UNIVERSITY OF RUHUNA

Faculty of Technology

Bachelor of Information and Communication Technology
Level II (Semester I)

CRIPTIXZ

Group Members-

- **A.P.G.H.Pramoda – TG_2023_1715**
- **A.M.I.T.B.Adhikari - TG_2023_1758**
- **J.L.A.L.Sewwandi - TG_2023_1732**
- **N.V.T.J.Gunasekara – TG_2023_1742**

SOFTWARE REQUIREMENTS

SPECIFICATION (SRS)

Faculty of Technology Management System

TABLE OF CONTENTS

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 References
- 1.5 Overview

2. Overall System Description

- 2.1 Product Perspective
- 2.2 Product Functions (High-Level Summary)
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 Assumptions and Dependencies

3. Functional Requirements

- 3.1 User Authentication and Access Control
  - 3.1.1 Login
  - 3.1.2 Profile Management
- 3.2 Admin Functions
  - 3.2.1 User Management
  - 3.2.2 Course Management
  - 3.2.3 Notice Management
  - 3.2.4 Timetable Management
- 3.3 Lecturer Functions

3.3.1 Course Materials
3.3.2 Mark Management
3.3.3 Student Data Viewing
3.4 Technical Officer Functions
3.4.1 Attendance Management
3.4.2 Medical Record Management
3.4.3 Viewing
3.5 Undergraduate Functions
3.6 Attendance Requirements (Detailed)
3.7 Marks and Grading Requirements (Detailed)
3.7.1 CA Eligibility
3.7.2 Overall Eligibility
3.7.3 Grading
3.7.4 GPA Calculation
3.7.5 Marks Views
4. Non-Functional Requirements
4.1 Usability
4.2 Performance
4.3 Security
4.4 Reliability
4.5 Maintainability
4.6 Scalability
5. System Constraints
6. Database Requirements
7. External Interface Requirements
7.1 User Interface
7.2 Database Interface
7.3 Software Interface
8. Appendix
8.1 Deliverables Schedule
8.2 Evaluation Criteria
8.3 Required OOP Concepts Checklist

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Faculty of Technology Management System, developed as part of the ICT2132 Object Oriented Programming Practicum Mini Project. The system is designed to digitize and manage key academic and administrative operations within the Faculty of Technology, University of Ruhuna.

This document is intended for use by the development team (Group B08) and serves as the authoritative reference for system design, implementation, and evaluation.

1.2 Scope

The Faculty of Technology Management System is a desktop-based application built using Java (GUI) and MySQL. The system will manage:

- User Profiles for four roles: Admin, Lecturer, Technical Officer, and Undergraduate
- Course Details including materials uploaded by lecturers
- Undergraduate Marks including continuous assessments and final examinations
- Undergraduate Attendance with theory and practical component tracking
- Medical Records and approval workflows
- Notices for all users
- Timetables for departments

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
CA	Continuous Assessment (internal marks)
SGPA	Semester Grade Point Average
CGPA	Cumulative Grade Point Average
UGC	University Grants Commission (Sri Lanka)
OOP	Object Oriented Programming
FR	Functional Requirement
NFR	Non-Functional Requirement
GUI	Graphical User Interface
T	Theory component of a course module

Term	Definition
P	Practical component of a course module
1T	One credit of Theory (2 hours per session)
1P	One credit of Practical (2 hours per session)
Batch-Missed	Students who missed a semester and rejoined a later batch
Repeat Student	Students repeating a semester/module due to failure

1.4 References

- ICT2132 Object Oriented Programming Practicum Mini Project Brief – University of Ruhuna (2026)
- UGC Commission Circular No. 12-2024 – Grading Guidelines
- University of Ruhuna – Faculty of Technology Academic Timetable (Current Semester)

1.5 Overview

This SRS is organized as follows: Section 2 provides an overall system description. Section 3 details all functional requirements by user role. Section 4 covers non-functional requirements. Section 5 specifies system constraints. Section 6 describes the database requirements. Section 7 addresses external interface requirements.

2. Overall System Description

2.1 Product Perspective

The Faculty of Technology Management System is a standalone, multi-user desktop application. It is not a replacement for any existing university-wide ERP system, but rather a purpose-built solution for managing academic data within the faculty. The system will use a centralized MySQL database accessed by a Java Swing/JavaFX GUI application.

2.2 Product Functions (High-Level Summary)

- User authentication and role-based access control
- Full CRUD operations on user profiles (by Admin)
- Course management with material uploads (by Lecturer)
- Attendance recording with theory/practical breakdown (by Technical Officer)
- Medical record submission and approval tracking
- Mark entry for continuous assessments and final exams (by Lecturer)
- Automatic eligibility calculation (CA ≥ 40% AND Attendance ≥ 80%)
- Grade computation per UGC Circular No. 12-2024
- SGPA and CGPA computation for each undergraduate
- Notice board management and timetable viewing

2.3 User Classes and Characteristics

User Role	Count (Minimum)	Characteristics
Admin	1	Full system access. Creates and maintains all user accounts, courses, notices, and timetables.
Lecturer	5+	Academic staff. Uploads marks, views student records, manages course materials.
Technical Officer	4+	Administrative staff. Records attendance and medical data for undergraduates.
Undergraduate	20+	Students including regular, repeat, and batch-missed students. Limited self-service access.

2.4 Operating Environment

- Platform: Desktop (Windows/Linux/macOS)
- Programming Language: Java (JDK 11 or higher recommended)
- GUI Framework: Java Swing or JavaFX
- Database: MySQL 8.0 or higher

- Database Connectivity: JDBC (MySQL Connector/J)

2.5 Design and Implementation Constraints

- The system must demonstrate the following OOP concepts in code: Classes & Objects, Inheritance, Abstraction, Polymorphism, Encapsulation, Error & Exception Handling, Database Handling, GUI
- All passwords must be stored securely (hashed)
- Usernames and passwords cannot be changed by non-admin users
- All marks must be stored and displayed out of 100
- Grading must strictly follow UGC Commission Circular No. 12-2024

2.6 Assumptions and Dependencies

- The current semester academic timetable is available and defines the course modules
- Both theory and practical components have exactly 15 sessions per module
- 1 credit of theory (1T) = 2 hours per session; 1 credit of practical (1P) = 2 hours per session
- The evaluation criteria (CA weightings) are provided by the course coordinators
- CA is the aggregate of internal assessments; final exam is a separate component

3. Functional Requirements

3.1 User Authentication and Access Control

3.1.1 Login

- FR-001: All users must log in using a unique username and password
- FR-002: The system must authenticate users and grant role-specific access
- FR-003: Failed login attempts must display an appropriate error message
- FR-004: Sessions must be role-bound; one active session per user

3.1.2 Profile Management

- FR-005: Admin can create, edit, and delete all user profiles
- FR-006: Admin can set and reset usernames and passwords for all users
- FR-007: Lecturers can update their own profile information EXCEPT username and password
- FR-008: Technical Officers can update their own profile information EXCEPT username and password
- FR-009: Undergraduates can update only their contact details (phone number, email, address) and profile picture

3.2 Admin Functions

3.2.1 User Management

- FR-010: Admin can create user accounts for Admin, Lecturer, Technical Officer, and Undergraduate roles
- FR-011: Admin can update any user's profile details
- FR-012: Admin can deactivate or delete user accounts
- FR-013: System must store at least 1 Admin, 5 Lecturers, 4 Technical Officers, and 20 Undergraduates (including repeat and batch-missed students)

3.2.2 Course Management

- FR-014: Admin can create new course modules with relevant details (code, name, credits, type: T/P/T+P)
- FR-015: Admin can assign lecturers to course modules
- FR-016: Admin can edit and deactivate course modules

3.2.3 Notice Management

- FR-017: Admin can create notices with title, body, date, and target audience

- FR-018: Admin can edit and delete notices
- FR-019: Notices must be visible to all relevant user roles

3.2.4 Timetable Management

- FR-020: Admin can create and maintain department timetables
- FR-021: Timetables must specify day, time slot, module, lecturer, venue, and type (T/P)
- FR-022: Admin can update and delete timetable entries

3.3 Lecturer Functions

3.3.1 Course Materials

- FR-023: Lecturer can upload study materials (files/links) for courses they are assigned to
- FR-024: Lecturer can modify existing course materials

3.3.2 Mark Management

- FR-025: Lecturer can upload marks for all assessment types (quizzes, assignments, mid-exams, practical assessments, final exam) for modules they coordinate
- FR-026: All marks must be entered out of 100
- FR-027: CA marks are computed based on evaluation criteria provided per module
- FR-028: The system must automatically calculate whether CA ≥ 40% (eligible for final exam)

3.3.3 Student Data Viewing

- FR-029: Lecturer can view complete undergraduate profile details
- FR-030: Lecturer can view attendance records (theory, practical, combined) for undergraduates
- FR-031: Lecturer can view medical records and their approval status for undergraduates
- FR-032: Lecturer can view CA marks and final marks per student per subject
- FR-033: Lecturer can view computed grades and GPA (SGPA and CGPA) per student
- FR-034: Lecturer can view eligibility status (attendance + CA) per student individually and as a batch summary

3.4 Technical Officer Functions

3.4.1 Attendance Management

- FR-035: Technical Officer can record attendance for each undergraduate per session per module
- FR-036: Attendance must distinguish between Theory (T) and Practical (P) session types

- FR-037: Each module has exactly 15 theory sessions and 15 practical sessions
- FR-038: The system must be able to show attendance for theory only, practical only, and combined
- FR-039: The system must calculate and display attendance percentage per student per module
- FR-040: The system must flag students with attendance below 80%, exactly 80%, and above 80%
- FR-041: Attendance summary must be viewable for the whole batch and for individual students

3.4.2 Medical Record Management

- FR-042: Technical Officer can record medical submissions made by undergraduates for absence days
- FR-043: Each medical record must store the student, affected date(s), reason, document reference, and approval status
- FR-044: Medical records must be markable as Approved or Pending/Rejected
- FR-045: Approved medicals must be factored into the effective attendance calculation

3.4.3 Viewing

- FR-046: Technical Officer can view notices
- FR-047: Technical Officer can view timetables of their own department

3.5 Undergraduate Functions

- FR-048: Undergraduate can view their own attendance records (theory, practical, combined) per module
- FR-049: Undergraduate can view their own medical record submissions and approval statuses
- FR-050: Undergraduate can view enrolled course details and uploaded materials
- FR-051: Undergraduate can view their own CA marks, final marks, grades, SGPA, and CGPA
- FR-052: Undergraduate can view their department timetable
- FR-053: Undergraduate can view notices

3.6 Attendance Requirements (Detailed)

The system must support all of the following attendance scenarios in its data and reporting:

Scenario	Attendance %	Has Medicals	Must Be Supported
S1	> 80%	No	Yes
S2	= 80%	No	Yes

Scenario	Attendance %	Has Medicals	Must Be Supported
S3	< 80%	No	Yes
S4	> 80%	Yes (Approved)	Yes
S5	< 80%	Yes (Approved)	Yes

- FR-054: Raw attendance % = (Sessions Attended / Total Sessions) x 100
- FR-055: Effective attendance % = ((Sessions Attended + Approved Medical Sessions) / Total Sessions) x 100
- FR-056: Eligibility threshold: Effective Attendance ≥ 80%

3.7 Marks and Grading Requirements (Detailed)

3.7.1 CA Eligibility

- FR-057: CA eligibility = CA total mark ≥ 40 (out of 100)
- FR-058: System must display CA eligibility status per student per module
- FR-059: Summary view must show CA eligibility for the whole batch

3.7.2 Overall Eligibility

- FR-060: A student is eligible to sit the final exam if and only if: Effective Attendance ≥ 80% AND CA ≥ 40%
- FR-061: System must display overall eligibility status per student and for the whole batch

3.7.3 Grading

- FR-062: Final marks must be computed per UGC Circular No. 12-2024
- FR-063: Grade boundaries and GPA values as per the UGC Circular must be applied

Grade	Marks Range	Grade Points
A+	85 – 100	4.00
A	75 – 84	4.00
A-	70 – 74	3.70
B+	65 – 69	3.30
B	60 – 64	3.00
B-	55 – 59	2.70
C+	50 – 54	2.30
C	45 – 49	2.00

Grade	Marks Range	Grade Points
C-	40 – 44	1.70
D+	35 – 39	1.30
D	30 – 34	1.00
E	0 – 29	0.00

Note: Grade boundaries above are based on UGC Circular No. 12-2024. The actual circular must be consulted for the authoritative values.

3.7.4 GPA Calculation

- FR-064: SGPA = Sum(Grade Points x Credits) / Sum(Credits) for modules in the semester
- FR-065: CGPA = Sum(Grade Points x Credits) / Sum(Credits) for all completed modules
- FR-066: SGPA and CGPA must be viewable per individual student and for the whole batch

3.7.5 Marks Views

- FR-067: System must show marks per subject, per individual student
- FR-068: System must show marks summary for the whole batch per subject
- FR-069: System must show final marks for individual students and the whole batch

4. Non-Functional Requirements

4.1 Usability

- NFR-001: The GUI must be intuitive and role-appropriate; users should be able to navigate without a manual after a brief orientation
- NFR-002: Error messages must be descriptive and guide the user toward resolution
- NFR-003: All data tables must support sorting and basic filtering

4.2 Performance

- NFR-004: Login must complete within 2 seconds under normal database load
- NFR-005: Any query returning batch-level data (e.g., all student grades) must return within 5 seconds
- NFR-006: The system must support concurrent access by multiple users without data corruption

4.3 Security

- NFR-007: User passwords must be stored as hashed values (e.g., bcrypt or SHA-256); plain text storage is not permitted
- NFR-008: Role-based access control must be strictly enforced at the application layer
- NFR-009: No user can access data or functions outside their role scope

4.4 Reliability

- NFR-010: The system must handle invalid inputs gracefully without crashing
- NFR-011: Database transactions must be atomic; partial updates must be rolled back on failure
- NFR-012: Exception handling must be implemented throughout and meaningful logs maintained

4.5 Maintainability

- NFR-013: Code must be organized using OOP principles (Classes, Inheritance, Abstraction, Polymorphism, Encapsulation)
- NFR-014: Code must be modular with clear separation of concerns (e.g., MVC or layered architecture)
- NFR-015: All public methods and classes must be appropriately documented (Javadoc)

4.6 Scalability

- NFR-016: The database schema must support additional course modules without structural changes
- NFR-017: New user roles or additional semesters can be added with minimal code changes

5. System Constraints

- The system must be implemented in Java using Object Oriented Programming principles
- The database must be MySQL
- The GUI must be implemented using Java Swing or JavaFX
- The project source code must be shared via a Git repository
- Each group member must individually demonstrate at least the following OOP concepts in their coded portion: Classes & Objects, Inheritance, Abstraction, Polymorphism, Encapsulation, Error & Exception Handling, Database Handling, GUI
- Minimum dataset constraints: 1 Admin, 5+ Lecturers, 4+ Technical Officers, 20+ Undergraduates (including repeat and batch-missed)
- Attendance data must cover all 5 specified scenarios (S1–S5 as above)
- All marks must be stored out of 100

6. Database Requirements

The MySQL database must store the following core entities and relationships:

Entity / Table	Key Attributes	Notes
users	user_id, username, password_hash, role, full_name, email, phone, address, department, profile_picture, is_active	Central table for all roles
departments	dept_id, dept_name	Faculty departments
courses	course_id, course_code, course_name, credits, type (T/P/T+P), dept_id, lecturer_id, is_active	Course modules
course_materials	material_id, course_id, lecturer_id, title, file_path, uploaded_at	Materials per course
enrollments	enrollment_id, student_id, course_id, semester, enrollment_status (regular/repeat/batch-missed)	Student-course link

Entity / Table	Key Attributes	Notes
timetable	timetable_id, dept_id, day, time_start, time_end, course_id, venue, session_type (T/P)	Department timetables
sessions	session_id, course_id, session_number, session_type (T/P), session_date	Individual sessions (15T + 15P per course)
attendance	attendance_id, session_id, student_id, status (present/absent)	Per-session attendance
medicals	medical_id, student_id, date_from, date_to, reason, document_ref, approval_status, reviewed_by, reviewed_at	Medical submissions
assessments	assessment_id, course_id, assessment_name, assessment_type, max_mark, weight_percent	CA component definitions
marks	mark_id, assessment_id, student_id, mark_obtained, entered_by, entered_at	Individual assessment marks
final_results	result_id, student_id, course_id, ca_total, ca_eligible, attendance_eligible, overall_eligible, final_exam_mark, final_total, grade, grade_points	Computed results per student per course
gpa_records	gpa_id, student_id, semester, sgpa, cgpa, computed_at	GPA summary per student per semester
notices	notice_id, title, body, created_by, created_at, target_role	Notice board

All foreign key relationships must be defined with appropriate referential integrity constraints. Indexes should be created on frequently queried columns such as student_id, course_id, and session_id.

7. External Interface Requirements

7.1 User Interface

- The GUI must use a consistent layout with a navigation menu reflecting the user's role
- Login screen must be the entry point for all users
- Dashboard must be role-specific, showing relevant quick stats (e.g., for lecturers: pending mark entries; for Technical Officers: today's attendance session)
- Tables displaying batch data must support column sorting
- Forms must validate inputs before submitting to the database

7.2 Database Interface

- Connectivity via JDBC using MySQL Connector/J
- Database connection parameters (host, port, database name, user, password) must be configurable (e.g., via a properties file)
- All database operations must use prepared statements to prevent SQL injection

7.3 Software Interface

- Java Development Kit (JDK 11 or later)
- MySQL Server 8.0 or later
- MySQL Connector/J (JDBC Driver)
- Optional: A build tool such as Maven or Gradle for dependency management

8. Appendix

8.1 Deliverables Schedule

Deliverable	Due Date	Submission Method
Software Requirements Specification (SRS)	1st March 2026	Upload to LMS (Mini Project section) – As Groups
UML Class Diagram + ER Diagram	8th March 2026	Upload to LMS (Mini Project section) – As Groups
Complete Mini Project + Report	26th April 2026	Report to LMS + Source Code to Git repository
Project Evaluation / Presentation	To be announced	15 min presentation + 10 min discussion

8.2 Evaluation Criteria

- Each group member must individually demonstrate use of all required OOP concepts in their coded portion
- Presentation: 15 minutes + 10 minutes Q&A per group
- Individual evaluation on personal coding contribution
- Report must include individual contribution section (1 page per member)

8.3 Required OOP Concepts Checklist

OOP Concept	Where Demonstrated (Example)
Classes and Objects	User, Lecturer, Student, Course, Attendance, Medical, Notice, Timetable classes
Inheritance	User (base) -> Admin, Lecturer, TechnicalOfficer, Undergraduate (subclasses)
Abstraction	Abstract User class with abstract methods (e.g., getDashboard())
Polymorphism	Method overriding in subclasses; e.g., each role overrides getPermissions()
Encapsulation	Private fields with public getters/setters in all model classes
Error & Exception Handling	Try-catch blocks for DB operations, input validation, custom exceptions
Database Handling	JDBC with PreparedStatement for all CRUD operations on MySQL
GUI	Java Swing/JavaFX forms, tables, dialogs, and navigation menus